Now I have every single file. Here is the complete implementation plan, step by step, with full explanation of every decision.

Overview: The Build Order

You always build foundations before the things that depend on them. The dependency chain flows like this:

```
Enums → Interfaces → Blackboard → Behaviors → Rules → Brain → Ant.cs refactor → Scene setup
```

Each step only depends on what came before it. This means you can implement and test each layer before moving to the next, and nothing breaks mid-way.

Step 1 — Expand `Enums.cs`

Why first: Every other file references these enums. They need to exist before anything else can compile.

What changes:

`AntRole` currently has `Worker` and `Soldier`. You need to add future types now so the interfaces that reference it are already future-proof:

```
AntRole { Worker, Soldier, Scout, Builder, Nurse }
```

`AntType` currently lives inside `Egg.cs` which is wrong — it's a global concept used by Queen, Nursery, and Egg. Move it into `Enums.cs` and delete it from `Egg.cs`:

```
AntType { Worker, Soldier, Scout, Builder, Nurse }
```

`WorkerJob` currently drives behavior decisions inside `Ant.cs`. Under the new system it becomes display-only — used by the debug UI to show what an ant is doing. Keep it but stop using it for logic. Add one new value:

```
WorkerJob { ..existing.., Any }
```

`Any` means "no preference assigned yet, figure it out yourself." This is the fallback state the Brain assigns when no rule claims an ant.

What to do with `AntRole` vs `AntType`: These serve different purposes. `AntType` is biological — what kind of ant this physically is (Worker, Soldier). `AntRole` is the same concept from the game logic side. They should stay in sync. For now they mirror each other exactly. In the future you could have one `AntType.Worker` ant that's promoted to a different role, but for now treat them as identical and map them 1:1 in `Nursery.cs`.

Step 2 — Create `IAntBehavior.cs`

Why second: Every behavior class and the Brain both depend on this interface. It must exist before either of them.

What this file is: A pure interface. No logic, no MonoBehaviour. Just a contract that every job must fulfill.

Fields and methods to define:

```
string BehaviorName
  → A readable name like "Dig", "CollectFood", "Guard"
  → Used only in the debug UI display
  → Not used for any logic decisions

AntRole[] EligibleRoles
  → An array of roles that are allowed to do this job
  → Behavior_Dig returns { Worker }
  → Behavior_Guard returns { Soldier }
  → Behavior_Idle returns { Worker, Soldier, Scout, Builder, Nurse } — everyone can idle
  → The Brain checks this before assigning — if the ant's role isn't in this list, it's skipped
  → This is how you enforce "Soldiers never collect food" without a single if/else anywhere

bool CanBeInterrupted(Ant ant)
  → The Brain calls this before reassigning
  → Returns false when the ant is committed mid-task
  → Each behavior implements its own definition of "committed"
  → Behavior_CollectFood: committed if carryingFoodAmount > 0 (can't drop food mid-carry)
  → Behavior_Dig: committed if ant is actively digging (digCooldown > 0 and at the wall)
  → Behavior_Idle: always true — idle ants are always reassignable
  → Behavior_Guard: always true — soldiers can be pulled off guard duty instantly

void OnAssigned(Ant ant)
  → Called exactly once when the Brain assigns this behavior to an ant
  → Used to reset state: clear the current path, null out old targets, reset timers
  → Prevents the ant from inheriting stale state from its previous job
  → Example: if an ant was collecting water and gets reassigned to digging, OnAssigned
    clears targetWater and currentPath so it doesn't try to pathfind to the old water source

void Tick(Ant ant)
  → Called every frame by the ant's Update()
  → This is where all the actual job logic lives
  → Gets passed the Ant reference so it can read position, set carry amounts, call movement helpers
  → Contains a mini state machine for each job (find target → move to it → collect → return)

bool IsComplete(Ant ant)
```

→ Called by the Brain periodically (not every frame — checked on the rebalance timer)
→ Returns true when the job has naturally finished
→ Example: Behavior_CarryEgg returns true when carriedEgg == null and targetEgg == null (meaning the egg was delivered and there are no more eggs to find)
→ When IsComplete returns true, the Brain re-evaluates this ant on the next pass

Why a pure interface instead of an abstract class: You want each behavior to be a clean, self-contained object with no shared base class overhead. An interface forces the contract without imposing any implementation. If you later want shared utilities between behaviors, you create a helper class that behaviors can reference — but the interface itself stays clean.

Step 3 — Create `IJobRule.cs`

Why third: Rules are what the Brain uses to make decisions. The Brain depends on this interface. It must exist before you write the Brain.

What this file is: Another pure interface. One implementation per job type.

Fields and methods to define:

IAntBehavior Behavior
→ A reference to the behavior this rule is scoring for
→ When the Brain decides "3 ants should dig", it reads this property to know what behavior to actually assign to those ants
→ The Rule and its Behavior are always paired — Rule_Dig owns Behavior_Dig

AntRole[] EligibleRoles
→ Mirrors the behavior's eligible roles
→ Exists on the rule as well so the Brain can check eligibility before even instantiating or fetching the behavior
→ In practice: Rule_Guard.EligibleRoles = { Soldier }, so the Brain's distribution loop skips all Worker ants when processing GuardRule

float DemandScore(ColonyBlackboard board)
→ The core of the intelligence system
→ Takes the Blackboard snapshot as input — reads nothing from the live game world directly
→ Returns a float: 0.0 means this job needs zero additional ants right now
→ Higher numbers mean higher urgency
→ The Brain collects all scores, sorts them, and distributes ants proportionally
→ This is what replaces the hardcoded priority waterfall
→ Full scoring logic explained in Step 5 (the rule implementations)

int MaxAntCap(int totalAnts)
→ A safety ceiling — even if demand is infinite, never assign more than this many ants
→ Expressed as a function of total ant count so it scales with colony size
→ Rule_Dig: returns Mathf.FloorToInt(totalAnts * 0.45f) — never more than 45% digging
→ Rule_Food: returns totalAnts — no cap, all ants can collect food in a crisis
→ Rule_Guard: returns int.MaxValue — all soldiers guard (they have no other option yet)

→ This prevents a single high-score job from monopolizing the entire colony

Why demand scores instead of priorities: A priority system (dig is priority 1, food is priority 2) is brittle. If food is critically low but there are also dig commands, a priority system always picks dig. A score system naturally balances — if food has a score of 30 and dig has a score of 10, food gets three times the ants. The actual numbers calibrate themselves to the situation.

Step 4 — Create `ColonyBlackboard.cs`

Why fourth: Both rules and the Brain read from the Blackboard. It must exist before either.

What this file is: A MonoBehaviour singleton that holds a data snapshot of the colony state. It does not make decisions. It only reads from game systems and stores the results cleanly.

Why a snapshot instead of live queries: If every rule queries `ResourceSpawner.Instance.activeFood.Count` directly during the Brain's rebalance, you're hitting the live list multiple times from multiple rules. The Blackboard reads it once, stores the count, and every rule reads the stored number. Cheaper, consistent, and when you add new data sources (threats, fog coverage) you add one field here and all rules see it immediately.

Fields to define:

```
// Resources stored in the colony
int foodStored
int waterStored
int mineralsStored

// Resources available in the world right now
int foodSourcesInWorld
int waterSourcesInWorld
int mineralSourcesInWorld

// Pending tasks
int pendingDigTargets       → read from DigCommandManager.targets.Count
int pendingEggs             → read from EggManager.Instance.availableEggs.Count

// Colony census
int totalAnts               → count of all registered ants
Dictionary<AntRole, int> antsByRole         → how many Worker, Soldier, etc.
Dictionary<string, int> antsByBehavior      → how many doing "Dig", "CollectFood", etc.
                                → keyed by BehaviorName string

// Phase 3+ (add fields here, all rules automatically see them)
List<Vector2> activeThreats       → empty list until threat system is built
int unexploredTileCount           → for Scout scoring
```

Method to define:

void Refresh(List<Ant> allAnts)
→ Called by the Brain at the start of each rebalance cycle
→ Reads ResourceStorage.Instance for stored amounts
→ Reads ResourceSpawner.Instance for world source counts
→ Reads DigCommandManager for pending target count
→ Reads EggManager for egg count
→ Rebuilds antsByRole and antsByBehavior by iterating allAnts
→ This is the only place in the codebase that queries all those systems

Why antsByBehavior is a Dictionary keyed by string: Because `BehaviorName` is a string defined on each behavior. This means you never have to update the dictionary structure when you add a new behavior — the new behavior's name just becomes a new key automatically.

Step 5 — Create the Behavior Classes (8 files)

Why fifth: Behaviors have no dependencies except `IAntBehavior`, `Ant`, and the game systems they call into (Pathfinder, GridManager, etc.). They can be written and tested independently.

Strategy for extraction: Every `XxxBehavior()` private method in `Ant.cs` becomes the `Tick()` method of a behavior class. The ant's fields that the behavior reads become parameters passed through the `Ant` reference. The ant's fields that the behavior writes are set directly on the ant object.

Important note about Ant field access: The behaviors need to read and write private fields on `Ant` like `carryingFoodAmount`, `targetFood`, `currentPath`. You have two options:

- Make those fields `public` — simple, slight loss of encapsulation
- Create a nested `AntState` struct that the ant exposes publicly — cleaner but more work

For this project, making the relevant fields `public` is the right call. It's a game, not a library. The behaviors are tightly coupled to the ant by design.

`Behavior_Idle.cs`

What it does: Makes the ant wander randomly near the queen when it has nothing to do.

EligibleRoles: All roles — any ant type can idle.

CanBeInterrupted: Always `true`. An idle ant is always reassignable.

OnAssigned: Picks a new wander target immediately so the ant doesn't freeze.

Tick: Direct copy of `IdleBehavior()` and `PickNewWanderTarget()` from `Ant.cs`. Move them here verbatim.

IsComplete: Always `false`. Idle never "completes" — the Brain just reassigns away from it when something better is available.

`Behavior_Return.cs`

What it does: Walks the ant back to the queen after finishing a dig task.

EligibleRoles: Worker only.

CanBeInterrupted: `false` while actively moving home. `true` once within 2 units of the queen.

OnAssigned: Clears currentPath so pathfinding starts fresh toward the queen.

Tick: Direct copy of `ReturnBehavior()` from `Ant.cs`. One addition: if new dig commands appear while returning, `IsComplete` returns true immediately so the Brain can reassign to digging on the next cycle.

IsComplete: Returns `true` when distance to queen is less than 2 units.

`Behavior_Dig.cs`

What it does: Walks the ant to a dig target and digs through walls toward it.

EligibleRoles: Worker only.

CanBeInterrupted: `false` when the ant is actively at the wall swinging (digCooldown > 0 and at frontier position). `true` when walking toward a dig target but hasn't reached the frontier yet — the Brain can pull the ant away and reassign before they commit to digging.

OnAssigned: Calls `DigCommandManager.ReleaseAnt(ant)` first to free any previous assignment, then clears currentPath.

Tick: Direct copy of `DiggerBehavior()` and its helpers (`FindNearestWalkableTo`, `DigTowardTarget`) from `Ant.cs`.

IsComplete: Returns `true` when `DigCommandManager.HasTargetForAnt(ant)` is `false` — meaning all targets are taken by other ants or completed.

Key detail: When the ant finishes a dig target, instead of directly setting `currentJob = WorkerJob.Returning`, it sets a flag `wantsToReturn = true`. The Brain sees `IsComplete` returning `true` and on the next cycle assigns `Behavior_Return`. This keeps the return behavior properly decoupled from the dig behavior.

`Behavior_CollectFood.cs`

What it does: Finds a food source, moves to it, collects it, returns it to storage.

EligibleRoles: Worker only.

CanBeInterrupted: `false` when `carryingFoodAmount > 0` — the ant is carrying food home. `true` when searching for food or moving toward it (hasn't collected yet).

OnAssigned: Nulls out `targetFood` and clears `currentPath` so the ant searches fresh.

Tick: Direct copy of `FoodCollectorBehavior()`, `FindFood()`, `CollectFood()`, `DeliverFood()` from `Ant.cs`.

IsComplete: Returns `true` when food has been delivered (`carryingFoodAmount == 0` after having collected) AND `targetFood == null`. The Brain then decides whether to reassign to food again or to something else.

Key subtlety — the hand-off problem: After delivery, if the Brain doesn't rebalance for another 3 seconds, the ant would just sit there. Solution: after delivering, the behavior sets itself to a "delivered" sub-state. In this sub-state, `IsComplete` returns `true`, which flags the ant for reassignment on the next Brain tick. Because the Brain ticks every 3 seconds but this check happens every frame inside the behavior, there's at most a 3-second delay before the ant gets a new job — acceptable.

`Behavior_CollectWater.cs` and `Behavior_CollectMinerals.cs`

What they do: Identical pattern to `Behavior_CollectFood` for their respective resources.

EligibleRoles: Worker only.

CanBeInterrupted: `false` when carrying their resource home.

Tick: Direct copy of the corresponding methods from `Ant.cs`.

Why separate classes instead of one `Behavior_CollectResource` with a type parameter: Type parameters would require casting and complicate the Brain's behavior assignment. Three separate classes are verbose but each is completely independent — changing water collection never risks breaking food collection.

`Behavior_CarryEgg.cs`

What it does: Finds an uncarried egg, picks it up, delivers it to the Nursery.

EligibleRoles: Worker only.

CanBeInterrupted: `false` when `carriedEgg != null` — the ant is physically carrying an egg. `true` when searching for an egg or moving toward one (hasn't picked up yet).

OnAssigned: Nulls out `targetEgg`. Does NOT null `carriedEgg` — if the ant was already carrying one when reassigned (shouldn't happen, but defensive), it continues delivery.

Tick: Direct copy of `EggCarrierBehavior()`, `FindEgg()`, `PickUpEgg()`, `DeliverEggToNursery()` from `Ant.cs`.

IsComplete: Returns `true` after egg delivery when `carriedEgg == null` and no new eggs exist.

`Behavior_Guard.cs`

What it does: Makes the soldier follow and orbit the queen.

EligibleRoles: Soldier only. This single line is what permanently separates soldier and worker behavior — no if/else in Ant.cs needed.

CanBeInterrupted: `true` always. When a threat system is added in Phase 3, you change this to `false` while actively engaging a threat. No other file needs to change.

OnAssigned: Clears currentPath.

Tick: Direct copy of `SoldierBehavior()` from `Ant.cs`.

IsComplete: Always `false`. Guarding never completes — the Brain keeps soldiers on this unless a higher-scoring rule (future: `Rule_Attack`) claims them.

Why this is the correct design for soldiers: Currently `SoldierBehavior()` is a dead-end private method that can never be reached by Workers because of the role check at the top of Update(). Under the new system, the role check lives in `EligibleRoles = { Soldier }`. Workers simply cannot be assigned `Behavior_Guard` because the Brain's allocation loop filters by eligibility. Clean, declarative, impossible to bypass accidentally.

Step 6 — Create the Rule Classes (6 files)

Why sixth: Rules depend on `IJobRule`, `IAntBehavior`, and `ColonyBlackboard`. All three now exist.

How to think about the scoring numbers: The absolute values don't matter — only the ratios between scores do. If food scores 25 and water scores 5, food gets roughly 5x as many ants as water. Start with rough numbers and tune in the Inspector.

`Rule_Dig.cs`

Behavior: `Behavior_Dig`
EligibleRoles: Worker

DemandScore logic:

```
if pendingDigTargets == 0        → return 0
if pendingDigTargets == 1        → return 15
if pendingDigTargets >= 2        → return 15 + (pendingDigTargets - 1) * 5
```

Each additional dig target adds 5 to urgency. Capped by `MaxAntCap` anyway.

MaxAntCap: `Mathf.FloorToInt(totalAnts * 0.45f)` — never let digging consume more than 45% of workers. You always need some collectors.

Why not 100% when dig targets exist: If all ants dig, nobody collects food, the colony starves, the Queen can't lay eggs, the colony shrinks. The cap enforces strategic balance even when the player sends many dig commands.

`Rule_Food.cs`

Behavior: `Behavior_CollectFood`
EligibleRoles: Worker

DemandScore logic:

```
if foodSourcesInWorld == 0        → return 0  (no point sending ants, nothing to collect)
if foodStored > 30                → return 3   (comfortable, passive collection)
if foodStored between 15–30       → return 12  (moderate, maintain workforce)
if foodStored between 5–15        → return 25  (urgent, increase collectors)
if foodStored < 5                 → return 50  (critical, maximum priority)
```

MaxAntCap: `totalAnts` (no cap — in a food crisis, everyone except committed ants should collect food).

Why food gets the highest possible score: Food is the survival currency. Without it the Queen can't lay eggs, the colony can't grow. Every other resource is secondary to survival.

`Rule_Water.cs` and `Rule_Minerals.cs`

Same pattern as Rule_Food with different thresholds because water and minerals are less immediately critical than food. Their scores should peak lower than food's peak so they never outbid a food crisis but still compete during normal times.

Water peak score: 30 (less critical than food at 50)
Minerals peak score: 15 (least critical for survival)

`Rule_CarryEgg.cs`

Behavior: `Behavior_CarryEgg`
EligibleRoles: Worker

DemandScore logic:

```
if pendingEggs == 0               → return 0
if pendingEggs >= 1               → return 20 per egg (capped by MaxAntCap)
```

Eggs need to reach the Nursery quickly — a neglected egg just sits there contributing nothing. But egg carrying is short-duration so you don't need many ants assigned at once.

MaxAntCap: `Mathf.Min(pendingEggs, Mathf.FloorToInt(totalAnts * 0.20f))` — one ant per egg but never more than 20% of the workforce.

Why eggs score 20: High enough to pull idle ants immediately but not so high that it pulls ants off critical food collection. Tune this.

`Rule_Guard.cs`

Behavior: `Behavior_Guard`
EligibleRoles: Soldier

DemandScore logic:

```
Always return 100
```

Soldiers always score maximum for guarding because they have nothing else to do yet. When `Rule_Attack` is added in Phase 3 with a score of 200 when threats exist, soldiers automatically switch to attacking without any changes to this rule. The higher score wins.

MaxAntCap: `int.MaxValue` — all available soldiers should guard.

Why return a constant score instead of 0: If you return 0, soldiers fall through to `Behavior_Idle` and wander randomly. That's wrong for soldiers — they should always be near the queen. A constant high score keeps them on guard duty permanently until something more urgent (threats) outbids it.

Step 7 — Create `ColonyBrain.cs`

Why seventh: The Brain depends on everything above — Blackboard, Rules, Behaviors, Ants. It must be last in the dependency chain.

What this file is: A MonoBehaviour singleton on its own GameObject in the scene. It owns the full list of registered rules and ants, drives the Blackboard refresh, and runs the allocation pass.

Fields:

```
float rebalanceInterval = 3f        → Inspector-tunable. How often to reallocate.
ColonyBlackboard blackboard         → reference to the Blackboard component
List<IJobRule> rules                → all registered rules
List<Ant> allAnts                   → all living ants (registered via RegisterAnt/UnregisterAnt)
```

Startup — RegisterRule(): In `Awake()`, create instances of all rule objects and register them. This is the only place in the codebase that explicitly lists all job types:

```
RegisterRule(new Rule_Dig())
RegisterRule(new Rule_CarryEgg())
```

```
RegisterRule(new Rule_Food())
RegisterRule(new Rule_Water())
RegisterRule(new Rule_Minerals())
RegisterRule(new Rule_Guard())
```

When you add `Rule_Scout` in Phase 3, you add one line here. That's the only change required to integrate a new job type.

RegisterAnt(Ant) / UnregisterAnt(Ant): Called by ants in `Start()` and `OnDestroy()`. The Brain maintains a live list of all ants. This replaces the need for `FindObjectsOfType<Ant>()` which is slow.

The Rebalance() method — detailed flow:

```
Step 1: Refresh Blackboard
  blackboard.Refresh(allAnts)
  → Blackboard reads all game systems and updates its snapshot

Step 2: Score all rules
  List<(IJobRule rule, float score)> scoredRules
  foreach rule in rules:
    float score = rule.DemandScore(blackboard)
    if score > 0: add to scoredRules
  Sort scoredRules by score descending
  → You now have an ordered list: most urgent job first

Step 3: Build the reassignable pool
  List<Ant> pool = new List<Ant>()
  foreach ant in allAnts:
    if ant.currentBehavior == null: add to pool (no behavior yet)
    else if ant.currentBehavior.CanBeInterrupted(ant): add to pool
    → Committed ants (carrying, mid-dig) are NOT in the pool
    → They finish their current task independently

Step 4: Allocate ants to rules
  foreach (rule, score) in scoredRules:
    int alreadyAssigned = blackboard.antsByBehavior[rule.Behavior.BehaviorName] (or 0)
    int cap = rule.MaxAntCap(blackboard.totalAnts)
    int needed = Mathf.Max(0, cap - alreadyAssigned)
    int toAssign = Mathf.Min(needed, pool.Count)

    for i in 0..toAssign:
      Ant ant = FindBestAntFromPool(pool, rule)
        → "Best" = closest ant to the relevant target (food source, dig target, etc.)
        → Fallback: just take the first eligible ant from pool
      AssignBehavior(ant, rule.Behavior)
      pool.Remove(ant)

Step 5: Assign idle to all remaining pool ants
  foreach ant in pool:
```

AssignBehavior(ant, idleBehavior)

AssignBehavior(Ant ant, IAntBehavior behavior):

if ant.currentBehavior?.BehaviorName == behavior.BehaviorName: return
→ Don't reassign if already doing this job. Avoids OnAssigned resetting a working ant.
ant.SetBehavior(behavior)
→ Calls behavior.OnAssigned(ant) then sets ant.currentBehavior = behavior

FindBestAntFromPool(pool, rule):

This is where you add smart targeting in the future. For now: filter pool by `rule.EligibleRoles`, return the first match. Later: for `Rule_Dig`, pick the ant closest to a dig target. For `Rule_Food`, pick the ant closest to a food source. This improves efficiency without changing the allocation logic at all.

Step 8 — Refactor `Ant.cs`

Why last: `Ant.cs` now delegates to behaviors. All behaviors must exist before you gut the ant's internal methods.

What gets deleted:

- `WorkerBehavior()` — replaced by the Brain's allocation + behaviors
- `SoldierBehavior()` — replaced by `Behavior_Guard`
- `IdleBehavior()`, `PickNewWanderTarget()` — moved to `Behavior_Idle`
- `DiggerBehavior()`, `DigTowardTarget()` — moved to `Behavior_Dig`
- `FoodCollectorBehavior()`, `FindFood()`, `CollectFood()`, `DeliverFood()` — moved to `Behavior_CollectFood`
- Same pattern for Water and Minerals
- `EggCarrierBehavior()`, `FindEgg()`, `PickUpEgg()`, `DeliverEggToNursery()` — moved to `Behavior_CarryEgg`
- `ReturnBehavior()` — moved to `Behavior_Return`

What stays in `Ant.cs`:

- All fields (carry amounts, target references, path state, speed, vision, timers)
- `FollowPathTo()`, `GetPath()`, `FindNearestWalkableTo()` — shared utilities used by all behaviors
- `UpdateVision()` — runs on stagger, independent of behavior
- `Start()` — registers with ColonyBrain, picks initial behavior
- `OnDestroy()` — unregisters from ColonyBrain and DigCommandManager
- `SetBehavior(IAntBehavior behavior)` — the assignment method

New fields to add:

csharp
public IAntBehavior currentBehavior // current assigned job

```
public WorkerJob displayJob          // for UI only — set by SetBehavior from behavior.BehaviorName
```

`Start()` changes:

```
Register with ColonyBrain: ColonyBrain.Instance.RegisterAnt(this)
Set initial behavior: SetBehavior(new Behavior_Idle())
→ Every ant starts idle. The Brain assigns real work within 3 seconds.
```

`Update()` becomes:

```
if (stagger frame) → UpdateVision()
currentBehavior?.Tick(this)
```

That's the entire update loop. Thirteen hundred lines of if/else chains replaced by two lines.

`SetBehavior()` implementation:

```
void SetBehavior(IAntBehavior newBehavior):
  currentBehavior?.OnExit(ant)      // optional: add OnExit to interface if cleanup is needed
  currentBehavior = newBehavior
  currentBehavior.OnAssigned(this)
  displayJob = MapBehaviorNameToEnum(newBehavior.BehaviorName)  // for UI
```

Field visibility changes: The behavior classes need to read and write fields that were previously private. Change these to `public`:

- `carryingFoodAmount`, `carryingWaterAmount`, `carryingMineralAmount`
- `targetFood`, `targetWater`, `targetMineral`
- `carriedEgg`, `targetEgg`
- `currentPath`, `pathIndex`, `pathCooldown`, `pathRefreshRate`
- `digCooldown`, `digRate`
- `wanderTarget`, `wanderTimer`

Step 9 — Scene Setup

What to do in the Unity Editor:

1. Create a new empty GameObject called `ColonyBrain`
2. Attach `ColonyBrain.cs` component
3. Create a new empty GameObject called `ColonyBlackboard`
4. Attach `ColonyBlackboard.cs` component
5. In `ColonyBrain` Inspector: assign the `ColonyBlackboard` reference, set `rebalanceInterval` (start with 3)
6. The `ColonyBrain.Awake()` registers all rules in code — no Inspector wiring needed for rules

7. `Nursery.SpawnAnt()` already sets `ant.role` — this now automatically determines which behaviors the ant is eligible for

What does NOT need changing in the scene:

- No existing GameObjects need modification
- No prefab changes needed
- ResourceSpawner, DigCommandManager, EggManager, GridManager — all unchanged

Step 10 — Debug UI (Recommended Before Tuning)

Why this matters: The demand scoring numbers need tuning. Without a display you're tuning blind.

What to add to `GameManager.cs`:

In the existing Update() that already shows food/water/mineral counts, add:

```
foreach behavior name in ColonyBrain's tracked behaviors:
  display: "Dig: 3 | Food: 4 [CRITICAL] | Guard: 2 | Idle: 0"
```

`ColonyBlackboard.antsByBehavior` gives you exactly this data. One dictionary, one display loop. No other changes needed.